GS CANVAS · YOUR VISION, OUR CANVAS

TECHNICAL ROADMAP

Backend · Database · Server · Admin Panel

A complete guide to evolving GS Canvas from a frontend-only Next.js app into a production-grade ecommerce platform.

PREPARED FOR

Nandakishore M — GS Canvas Project

DOCUMENT VERSION

v1.0 · May 2026

TECH STACK

Next.js 15 · PostgreSQL · Node.js · Cloudinary

TABLE OF CONTENTS

1 CURRENT ARCHITECTURE OVERVIEW

GS Canvas is currently a **frontend-only Next.js 15 application** deployed on Vercel. It has a beautiful customer-facing UI with Shop, Custom Studio, Collections, and About pages. However, it lacks a backend, database, user auth, and payment integration — all essential for a live ecommerce business. The steps below convert it into a full-stack production platform.

Frontend	Next.js 15 + Tailwind CSS (deployed on Vercel)
Hosting	Vercel (frontend only — no server/DB connected)
Missing	Backend API, Database, Auth, Payments, Admin Panel, Image uploads
Repo	GitHub (nandakishorem05/videosnip or gs-canvas repo)
Domain	gs-canvas.vercel.app (needs custom domain for production)

2 BACKEND DEVELOPMENT — API ROUTES & SERVER LOGIC

Since you're already on Next.js 15, use **Next.js API Routes** (App Router with Route Handlers) as your backend. This means zero extra server setup — your backend lives inside the same project under **app/api/** and deploys automatically to Vercel.

API Routes to Build

Route	Method	Purpose
/api/products	GET	Fetch all products from DB
/api/products/[id]	GET	Fetch single product details
/api/products	POST	Admin: Create new product
/api/products/[id]	PUT/DELETE	Admin: Edit or delete product
/api/orders	POST	Create new order after payment
/api/orders/[id]	GET	Fetch order details
/api/orders/all	GET	Admin: List all orders
/api/auth/register	POST	User registration
/api/auth/login	POST	User login (JWT issue)
/api/upload/image	POST	Upload image to Cloudinary
/api/upload/cover	POST	Upload homepage cover/banner
/api/custom-orders	POST	Submit custom artwork request
/api/payments/razorpay	POST	Create Razorpay order
/api/payments/verify	POST	Verify payment signature

Recommended Libraries

- **Prisma ORM** — Type-safe database queries, auto-generates schema
- **bcryptjs** — Password hashing for user auth
- **jsonwebtoken** — JWT generation & verification
- **zod** — Request body validation
- **multer / formidable** — Handle multipart form data for file uploads

3 DATABASE DESIGN — POSTGRESQL SCHEMA

Use **PostgreSQL** as your primary database. Host it on **Supabase** (free tier available, includes a REST API, auth, and storage as a bonus). Connect via **Prisma ORM** inside Next.js.

Core Tables

■ USERS table

- id (UUID, PK)
- name (VARCHAR)
- email (VARCHAR, unique)
- password_hash (TEXT)
- role (ENUM: customer | admin)
- created_at (TIMESTAMP)
- phone (VARCHAR)
- address (JSONB)

■ PRODUCTS table

- id (UUID, PK)
- title (VARCHAR)
- description (TEXT)
- price (DECIMAL)
- category (ENUM: poster | canvas | sketch | custom)
- sizes (JSONB — array of sizes)
- images (JSONB — array of Cloudinary URLs)
- is_active (BOOLEAN)
- stock (INT)
- created_at (TIMESTAMP)

■ ORDERS table

- id (UUID, PK)
- user_id (FK → users)
- status (ENUM: pending | processing | shipped | delivered | cancelled)
- total_amount (DECIMAL)
- shipping_address (JSONB)
- payment_id (VARCHAR)
- payment_status (ENUM: paid | failed | refunded)
- items (JSONB — product snapshots)
- created_at (TIMESTAMP)

■ CUSTOM_ORDERS table

- id (UUID, PK)
- user_id (FK → users)
- image_url (TEXT — Cloudinary)
- style (ENUM: sketch | canvas | poster)
- size (VARCHAR)
- notes (TEXT)
- status (ENUM: received | in_progress | ready | delivered)

- price_quote (DECIMAL)
- created_at (TIMESTAMP)

■ BANNERS / COVERS table

- id (UUID, PK)
- image_url (TEXT — Cloudinary URL)
- title (VARCHAR)
- subtitle (TEXT)
- cta_text (VARCHAR)
- cta_link (VARCHAR)
- is_active (BOOLEAN)
- sort_order (INT)
- created_at (TIMESTAMP)

■ MEDIA_LIBRARY table

- id (UUID, PK)
- url (TEXT — Cloudinary URL)
- public_id (TEXT)
- type (ENUM: product | cover | banner | custom)
- uploaded_by (FK → users)
- uploaded_at (TIMESTAMP)

Prisma Setup (Quick Start)

```
npm install prisma @prisma/client
npx prisma init --datasource-provider postgresql
# Add DATABASE_URL to .env.local (Supabase connection string)
npx prisma db push # Push schema to Supabase
npx prisma generate # Generate Prisma client
```

4 IMAGE & FILE STORAGE — CLOUDINARY

Cloudinary is the perfect fit for GS Canvas — it handles product image uploads, cover page images, and custom artwork files. It provides automatic image optimization, transformations (resize, crop, format conversion), and a CDN for fast delivery.

What You'll Use Cloudinary For

- **Product images** — Multi-image upload per product, auto-resized
- **Cover/banner images** — Admin uploads hero images from the panel
- **Custom studio uploads** — Customers upload their own photos for artwork
- **Media library** — Admin browses all uploaded assets in one place

Integration Steps

```
npm install cloudinary next-cloudinary
```

Add to **.env.local**:

```
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
```

For the admin panel, use the **CldUploadWidget** from next-cloudinary — it gives a drag-and-drop upload UI with instant preview. After upload, save the Cloudinary URL to your PostgreSQL database.

Free tier	25 GB storage + 25 GB bandwidth/month — plenty to start
Transformations	Auto-resize to thumbnail, product card, fullscreen sizes
Folders	Organize: /products/, /covers/, /custom-orders/, /media/
Security	Signed uploads from server-side API — prevents unauthorized uploads

5 AUTHENTICATION SYSTEM

Use **NextAuth.js v5 (Auth.js)** — the standard for Next.js authentication. It handles sessions, JWT tokens, and social logins. You'll have two roles: **customer** (shop, order, upload custom artwork) and **admin** (full panel access).

Auth Providers to Enable

- **Email + Password** — Credentials provider (primary)
- **Google OAuth** — 'Sign in with Google' (increases conversion)
- **OTP via email** — Optional, for passwordless login

Role-Based Access Control (RBAC)

Protect admin routes using middleware. Any request to **/admin/*** checks the session token for role = 'admin'. Regular customers are redirected to the homepage.

```
// middleware.ts — Protect admin routes
export { default } from 'next-auth/middleware'
export const config = { matcher: ['/admin/:path*'] }
```

Setup

```
npm install next-auth@beta @auth/prisma-adapter
```

6 PAYMENT INTEGRATION — RAZORPAY

Use **Razorpay** for payment processing — it supports UPI, cards, net banking, and wallets in India. The flow is: frontend calls your API → API creates a Razorpay order → user completes payment in Razorpay popup → webhook confirms success → order saved to DB.

Payment Flow

- **Step 1:** Customer clicks 'Buy Now' → cart total sent to /api/payments/razorpay
- **Step 2:** API creates Razorpay order (server-side, secure)
- **Step 3:** Frontend opens Razorpay checkout popup with order ID
- **Step 4:** Customer pays → Razorpay sends payment_id + signature
- **Step 5:** Frontend calls /api/payments/verify → server verifies HMAC signature
- **Step 6:** On success → order created in DB, confirmation email sent

Setup

```
npm install razorpay
RAZORPAY_KEY_ID=rzp_test_xxxxxx
RAZORPAY_SECRET=your_secret_here
```

Webhooks: Configure Razorpay webhook → /api/payments/webhook to handle payment failures, refunds, and disputes automatically.

7 ORDER MANAGEMENT SYSTEM

The order management system tracks every purchase from payment to delivery. Customers can view their orders, and admins can update statuses and trigger email notifications.

Order Lifecycle

pending	Order placed, payment initiated
processing	Payment verified, artwork being prepared
shipped	Order handed to courier (tracking number added)
delivered	Confirmed delivered to customer
cancelled	Cancelled by customer or admin
refunded	Refund processed via Razorpay

Email Notifications

Use **Nodemailer + Gmail SMTP** or **Resend** (recommended — free 3k emails/month):

- Order confirmation email with summary + amount
- Shipping notification with tracking link
- Delivery confirmation
- Custom order status updates

```
npm install resend
```

8 ADMIN PANEL — COMPLETE FEATURE GUIDE

The admin panel lives at **/admin** and is accessible only to users with role = 'admin'. Build it as a set of protected Next.js pages with a sidebar layout. It gives you full control over products, orders, images, covers, and customers.

8.1 Dashboard (Homepage of Admin Panel)

- Total revenue (today / this week / this month)
- New orders count with status breakdown
- Total customers, new signups this week
- Pending custom artwork requests
- Quick action buttons: Add Product, View Orders, Upload Cover

8.2 Product Management

Full CRUD for all products in the shop.

- **Add Product:** Title, description, price, category, sizes (checkboxes), stock count
- **Image Upload:** Drag-and-drop multi-image upload via Cloudinary widget (up to 6 images/product)
- **Preview:** See how the product card looks before publishing
- **Toggle Active:** Show/hide products without deleting them
- **Bulk actions:** Delete multiple, mark as out-of-stock, apply discount

Product Form Fields:

Title	Short product name (e.g. 'Golden Sunset Canvas')
Category	Dropdown: Wall Poster | Canvas Print | Custom Sketch
Price (■)	Original price + optional sale price
Sizes	Multi-select: 8×10, 12×16, 18×24, 24×36 (custom too)
Stock	Number input per size variant
Description	Rich text editor (use react-quill or tiptap)
Images	Cloudinary widget — drag, reorder, set primary image
Tags	Searchable tags for SEO: modern, minimal, abstract...

8.3 Cover Page & Banner Management ★ (Your Main Request)

This is the section where you control what appears on the homepage hero, collection banners, and promotional strips — without touching any code.

- **Upload Hero Cover:** Upload a full-width image for the homepage hero section
- **Set Headline & CTA:** Edit the title ('Your Vision Our Canvas'), subtitle, and button text from admin
- **Multiple Banners:** Add/remove/reorder multiple promo banners (sale, seasonal, new collection)
- **Live Preview:** See a mini-preview of how the banner looks before saving
- **Schedule Banners:** Set start/end date for a sale banner (auto shows/hides)
- **Mobile Cover:** Upload separate mobile-optimized cover image

How it works technically: Admin uploads image → Cloudinary stores it → URL + headline text saved to 'banners' table in PostgreSQL → Homepage fetches active banner at build time (ISR every 60s). No code deploy needed to change the cover.

8.4 Media Library

- Grid view of all uploaded images (products, covers, custom orders)
- Filter by type: Products | Covers | Customer Uploads
- Copy Cloudinary URL to clipboard in one click
- Delete unused images (removes from Cloudinary + DB)
- Search by filename or upload date

8.5 Order Management

- Table of all orders: ID, customer name, date, amount, status
- Filter by status, date range, payment method
- Click any order → full detail: items, shipping address, payment info
- Update order status (Processing → Shipped → Delivered) with one click
- Add tracking number → triggers email to customer automatically
- Export orders as CSV (for accounting, courier handover)

8.6 Custom Artwork Orders

- See all custom studio submissions with the uploaded customer image
- View requested style, size, and any notes from the customer
- Set a price quote and send it to the customer via email
- Update status: Received → In Progress → Ready → Delivered
- Download the original customer image for your artist/printer

8.7 Customer Management

- List of all registered customers with order count and total spend
- View individual customer profile: orders, addresses, custom requests
- Search by name, email, or phone

8.8 Settings / Configuration

- Store name, logo, contact email, WhatsApp number
- Shipping rates (flat rate, free above ■X threshold)
- Tax settings (GST %age)
- Manage admin users (add/remove admins)
- Email template customization (order confirmation copy)

Recommended UI Library for Admin Panel

Library	shadcn/ui + Tailwind CSS (already used in your project)
Charts	Recharts or Chart.js for dashboard graphs
Tables	TanStack Table (React Table v8) — sortable, filterable, paginated

Rich Text	Tiptap editor for product descriptions
Upload	next-cloudinary CldUploadWidget — drag-and-drop, instant preview

9 DEPLOYMENT & HOSTING STRATEGY

Frontend + API	Vercel — Already deploying there. API routes deploy automatically. Free tier is sufficient.
Database	Supabase — Free PostgreSQL (500MB), built-in dashboard, real-time, REST API auto-generated
Image Storage	Cloudinary — Free tier: 25GB storage, 25GB bandwidth/month
Email	Resend — 3,000 free emails/month, Next.js integration with 1 package
Payments	Razorpay — No monthly fee, 2% per transaction, instant settlements
Auth	NextAuth.js — Self-hosted in your Vercel app, no extra cost
Custom Domain	Buy domain on Namecheap/GoDaddy (~■800/yr), connect to Vercel in 2 clicks
Monitoring	Vercel Analytics (free) — page views, performance, Web Vitals

10 PHASE-WISE IMPLEMENTATION ROADMAP

Follow these phases in order. Each phase builds on the previous.

Phase	Goal	Duration	Priority
Phase 1	Database setup (Supabase + Prisma), connect DB to Next.js, migrate static product data to DB	1 week	**High**
Phase 2	User authentication — NextAuth, register/login pages, JWT, role-based middleware	1 week	**High**
Phase 3	Cloudinary integration — product image uploads, /api/upload route, save URLs to DB	3–4 days	**High**
Phase 4	Admin Panel — Dashboard, Product CRUD, Cover/Banner upload, Media Library	2 weeks	**High**
Phase 5	Cart system — localStorage cart → DB cart sync for logged-in users	3–4 days	**High**
Phase 6	Razorpay payment integration — order creation, verify webhook, order DB records	1 week	**High**
Phase 7	Order management — customer order history, admin order dashboard, status updates	1 week	**Medium**
Phase 8	Email notifications — Resend integration, order confirmation, shipping alerts	2–3 days	**Medium**
Phase 9	Custom artwork orders — submission form, admin review, pricing, status tracking	1 week	**Medium**
Phase 10	SEO, sitemap, performance optimization, custom domain, go live	3–4 days	**Low**

Total estimated timeline: 8–10 weeks working part-time (4–5 hrs/day).

11 FULL TECH STACK SUMMARY

Layer	Technology	Purpose	Cost
Frontend	Next.js 15 + Tailwind	UI, routing, SSR/ISR	Free
Backend API	Next.js Route Handlers	REST API (lives in same app)	Free
ORM	Prisma	Type-safe DB queries	Free
Database	PostgreSQL on Supabase	All data: products, orders...	Free (500MB)
Auth	NextAuth.js v5	Login, sessions, roles	Free
Images	Cloudinary	Upload, store, serve images	Free (25GB)
Payments	Razorpay	UPI, cards, net banking	2% per txn
Email	Resend	Transactional emails	Free (3k/mo)
Deployment	Vercel	Hosting, CI/CD, analytics	Free tier
Domain	Namecheap / GoDaddy	Custom domain (e.g. gscanvas.in)	~■800/yr

12 ESTIMATED COST BREAKDOWN

GS Canvas can be run at **near-zero cost** in the early stage using free tiers. Below is a realistic estimate for startup and growth stages.

Startup Stage (0–50 orders/month) — ■0 to ■800/yr

Service	Plan	Cost
Vercel	Hobby (free)	■0/mo
Supabase	Free tier (500MB DB)	■0/mo
Cloudinary	Free (25GB)	■0/mo
Resend	Free (3k emails)	■0/mo
NextAuth	Self-hosted	■0/mo
Razorpay	2% per transaction	■0 fixed
Domain	gscanvas.in or .com	~■800/yr
TOTAL		~■67/mo

Growth Stage (100+ orders/month) — ■2,000–5,000/mo

- Vercel Pro: ■1,800/mo (faster builds, more bandwidth)
- Supabase Pro: $25/mo = ~■2,100/mo (8GB DB, more connections)
- Cloudinary Starter: $89/mo only if you exceed 25GB (unlikely early on)
- Resend Pro: $20/mo for 50k emails (needed at scale)

Next Step to Take Right Now: Set up Supabase (free account), run npx prisma init in your project, and define the products and users tables first. Once data flows from DB to your shop page, the rest of the stack falls into place naturally.

Built with care for GS Canvas · Nandakishore M · 2026